

The Trained Pointer

and how it relates to the Next-POI Engine

Smart Visit Module — companion note

Short answer — is it similar to your next-POI engine? Yes, to a large degree. The Trained Pointer *reuses the very same building blocks*: the 2-layer GCN over the hybrid POI graph, the learnable user embedding, a GRU of the same size, and an output distribution over the *whole* POI vocabulary. It is decoded with the *same* loop-free, reserved-end discipline as Frozen Rollout. It differs in three places: (1) the **output head** (an inner-product *pointer* instead of an MLP), (2) what the **GRU does** (it *decodes the output route* autoregressively from the query, instead of *encoding the input history*), and (3) the **training signal** (one whole-trajectory example per trip, instead of one example for every prefix). Point (3) is why it loses to Frozen Rollout.

1 The two models at a glance

Both models are built from the *same* encoder stack and only diverge at the “head” and in how they are trained and rolled out. Concretely (file references in `src/`):

- **Next-POI engine** — `src/models/next_poi.py` (`NextPOIModel`). Phase 1. Predicts the *single next* POI from a session prefix.
- **Trained Pointer** — `src/itinerary/pointer_model.py` (`PointerItineraryModel`). Phase 2, the “integrated” alternative. Generates a *whole ordered route* from a query.
- Both import the *identical* encoder modules `POIGraphEncoder` and `ContextEncoder` from `src/models/componen`. Default dimensions match: $d_p = 128$ (POI), $d_u = 64$ (user), $d_c = 32$ (context), $d_h = 128$ (GRU).

2 Recap: the Next-POI Engine (Phase 1)

A 2-layer GCN turns a learnable POI table into graph-aware features $H = \text{GCN}(\text{graph}) \in \mathbb{R}^{|V| \times d_p}$. For each visited POI p_t in the session, its feature row H_{p_t} is concatenated with a context vector $c_t = \text{ContextMLP}(\Delta d_t, \Delta t_t)$; a GRU reads the whole prefix and its final state h_T is combined with the user embedding e_u and scored by an MLP head over all $|V|$ POIs:

$$x_t = [H_{p_t} \parallel c_t], \quad h_T = \text{GRU}(x_1, \dots, x_T), \quad (1)$$

$$z = [h_T \parallel e_u], \quad \hat{y} = \text{MLP}(z) \in \mathbb{R}^{|V|}, \quad P(\text{next}) = \text{softmax}(\hat{y}). \quad (2)$$

It is trained with cross-entropy on *every prefix* of every session (a session of length L yields $L - 1$ training pairs — *dense* supervision), early-stopped on validation HR@10.

3 The Trained Pointer (Phase 2), step by step

The pointer keeps the GCN encoder H and the user embedding e_u , but instead of *reading* a history it *writes* a route. Figure 1 shows the data flow.

1. Encode (shared with Phase 1). $H = \text{GCN}(\text{graph}) \in \mathbb{R}^{|V| \times d_p}$ and $e_u \in \mathbb{R}^{d_u}$ — the same modules and weights-shape as the engine.

2. Initialise the decoder from the query. The query fixes a start s , an end e and a user u . These seed the decoder’s hidden state:

$$h_0 = \tanh(W_{\text{init}} [H_s \parallel H_e \parallel e_u]) \in \mathbb{R}^{d_h}. \quad (3)$$

This is something the engine never does — the engine has no notion of a fixed end.

3. Decode the route autoregressively. At step t the decoder input is the *previous* POI’s GCN feature (plus the $(\Delta d, \Delta t)$ context in the B-v2 variant), the GRU advances its state, and the state is projected to a *query vector* q_t :

$$g_t = H_{p_{t-1}} (\parallel c_t \text{ if context}), \quad h_t = \text{GRU}(g_t, h_{t-1}), \quad q_t = W_o [h_t \parallel e_u] \in \mathbb{R}^{d_p}. \quad (4)$$

4. Point at a POI (the key difference). Instead of an MLP head with its own per-POI weights, the score of POI v is just the *inner product* of the query vector with that POI’s GCN feature:

$$\boxed{\ell_t(v) = \langle q_t, H_v \rangle} \quad \ell_t = q_t H^\top \in \mathbb{R}^{|V|}. \quad (5)$$

The output is therefore tied directly to the graph features — there is no separate classification matrix to learn.

5. Train on whole trajectories; decode loop-free. Training is teacher-forced sequence cross-entropy over the *whole* ordered trajectory — one example per session (`ItinerarySeqDataset`), early-stopped on validation pairs-F1. At inference the route is rolled out with a **no-revisit mask** (visited POIs set to $-\infty$) and a **reserved end** (the last step is forced to e), greedy or beam — exactly the discipline Frozen Rollout applies to the frozen engine.

4 Side by side

Figure 2 lines the two pipelines up; **green** marks the modules that are **shared**, **blue** the engine-specific head, **orange** the pointer-specific head.

5 Why the Pointer does *not* beat Frozen Rollout

On Foursquare NYC (length- ≥ 3 test, $n = 2,880$), decoding the *frozen* engine (**Frozen Rollout**) actually *beats* the purpose-trained pointer:

Method	Type	pairs-F1
Frozen Rollout (greedy)	decoupled	0.289
Frozen Rollout (beam 3)	decoupled	0.290
Trained Pointer v1 (no context)	integrated	0.259
Trained Pointer v2 (+ context)	integrated	0.261

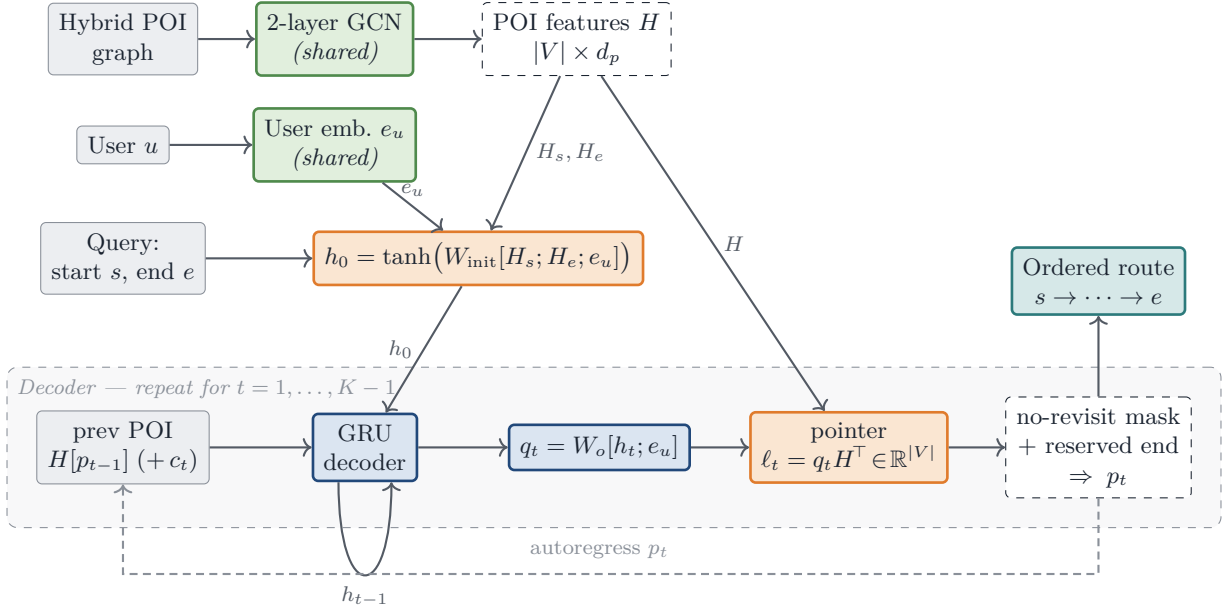
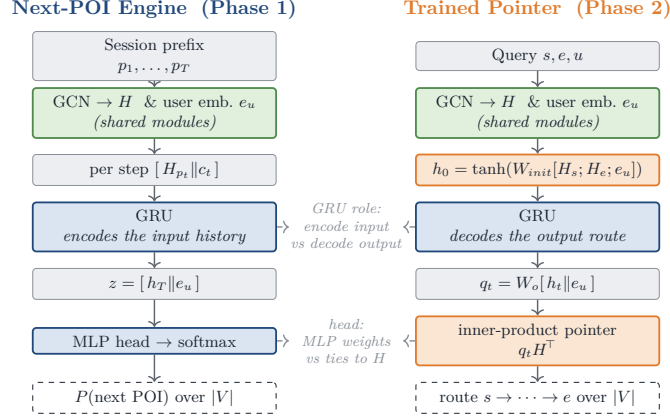


Figure 1: Architecture of the **Trained Pointer**. **Green** marks the modules shared with the next-POI engine (the GCN encoder and the user embedding). The query (s, e, u) initialises the GRU *decoder*; at each step the decoder state becomes a query vector q_t whose *inner product* with every POI feature H_v produces the next-POI scores; the route is rolled out loop-free with a reserved end. (e_u also enters q_t through $W_o[h_t; e_u]$.)

The decisive reason is **supervision density**. The engine learns from every prefix of every session ($\sim 10^5$ next-POI examples on NYC); the pointer sees only *one* whole-trajectory example per session — far fewer gradient signals, and it trains its encoder from scratch rather than reusing a converged one. Two secondary effects: the v1 pointer conditions only on the previous POI’s feature (adding $(\Delta d, \Delta t)$ in v2 recovers a little), and beam search barely helps either model (+0.0015), showing the limit is the *scorer*, not the search. The honest lesson: *naively training a dedicated itinerary model does not automatically beat cleverly decoding a strong next-POI model*.

In one line. The Trained Pointer is your next-POI engine’s encoder (GCN + user embedding) re-wired into a query-initialised, autoregressive route *decoder* with an inner-product head — same parts, different head, sparser training — and that sparser training is exactly why the simple decoupled Frozen Rollout wins. A lighter sibling of this pointer is used for the Flickr literature benchmark (`src/flickr/pointer.py`).



Training signal (the decisive difference). dense: one example *per prefix* ($\sim 10^5$ on NYC) vs sparse: *one whole trajectory* per session $\Rightarrow$ supervision-density gap

Figure 2: The next-POI engine and the Trained Pointer side by side. **Green** = modules shared by both (GCN encoder + user embedding); **blue** = the engine’s MLP head; **orange** = the pointer-specific parts (query-initialised decoder and inner-product head). Same encoder, different head, and — the decisive difference — a far sparser training signal.

	Next-POI engine	Trained Pointer
GCN graph encoder	✓ POIGraphEncoder	✓ <i>same</i> POIGraphEncoder
User embedding e_u	✓ $d_u = 64$	✓ <i>same</i> ($d_u = 64$)
$(\Delta d, \Delta t)$ context	✓ always	✓ optional (B-v2 only)
GRU	encodes the <i>input</i> history	decodes the <i>output</i> route (autoregressive)
Hidden init	zeros (reads a prefix)	$h_0 = \tanh(W_{init}[H_s; H_e; e_u])$ from the query
Output head	MLP $\rightarrow$ softmax over $ V $	inner product $q_t H^\top$
End POI	not modelled	given; reserved at the last step
Training examples	every prefix ($L-1$ per session; $\sim 10^5$ on NYC)	one whole trajectory per session
Early-stop metric	validation HR@10	validation pairs-F1
Output	a ranked next POI	an ordered, loop-free route $s \rightarrow \dots \rightarrow e$

Table 1: What is shared (✓) and what differs between the two models.